

VARLIK ÇERÇEVESİ

Nesne-İlişkili Eşleme

Katmanlı yazılım mimarisinin işlendiği *Katmanların Tasarlanması* başlığında iş katmanındaki veri taşıyan sınıflar (BEC) ışığında veri erişim katmanı tasarlandığı ve iş varlıklarının soyut olarak veri tabanında nasıl saklanacağına veri erişim bileşenleri (DAC) ile yapıldığı anlatılmıştı.

Veri erişim katmanında yer alacak her veri erişim bileşenlerinin (DAC) her biri, **yaratma** (create), veri tabanında **kayıt** (record), **okuma** (read), **güncelleme** (update) ve **silme** (delete) işlemini gerçekleştirilecek şekilde yöntemlere sahiptir.

Günümüzde iş varlığı bileşenlerini (BEC) için bu CRUD yöntemlerini otomatik sağlayan ve en az kodlama ile eşleştiren teknolojiler ortaya çıkmıştır. Bu teknolojinin adı **nesne-ilişkili eşlemedir** (Object-Relational Mapping-ORM). Kısaca iş varlıklarımızı veri tabanındaki tablolardaki kayıtlarla eşler. Yani projemizde CRUD işlevleri için gerekli SQL komutları ORM tarafından otomatik olarak oluşturulur ve bu teknoloji sayesinde daha hızlı ve hatasız yazılım geliştirilir.

ORM, çok fazla programlama yapmadan, otomatik bir şekilde iş nesnelerinden ilişkisel veri tabanına veri depolamak için bir araçtır. Üç ana bölümden oluşur:

1. İş bileşenlerini içeren etki alanı sınıf nesneleri
2. İlişkisel veri tabanı nesneleri
3. Etki alanı nesnelerinin ilişkisel veri tabanı nesnelerine nasıl eşlendiğine dair eşleme bilgileri.

ORM, veri tabanı tasarımımızı iş bileşeni sınıf tasarımımızdan ayrı tutmamızı sağlar. Bu, uygulamayı sürdürülebilir ve genişletilebilir hale getirir. Ayrıca standart CRUD işlemini (Oluşturma, Okuma, Güncelleme ve Silme) otomatikleştirir, böylece geliştiricinin bunu manuel olarak yazmasına gerek kalmaz.

Veri erişimi için veri erişim katmanında ADO.Net kodunu yazmak ve yönetmek sıkıcı ve monoton bir iştir. Microsoft, uygulamanız için veri tabanı ile ilgili faaliyetleri otomatikleştirmek amacıyla **varlık çerçevesini** (Entity Framework-kısaca EF) geliştirmiştir.

Örnek Kurumsal Uygulama başlığında verilen örnekte ilk önce veri tabanında tablo oluşturulmuş, daha sonrasında bu tabloya veri ekleme ve veri çekme işlemi kodlanmıştır. Bu yaklaşıma **önce veri tabanı** (database first) adı verilir.

Önce veri tabanı yaklaşımında veri tabanı ile iş varlıklar (BEC) eşleşmesi için ilk önce veri tabanı tabloları tanımlanır ve sonrasında bu tablolar ile varlıklar **eşleştirilir** (mapping). Bu da yazılımın devreye alınacağı zaman veri tabanı yöneticisine bir iş çıkarır. Yazılımın devreye alınmasında ve sonraki uygulama sürümlerinde kodlamayı yapan yazılımcının veri tabanı yöneticisi ile çalışmasını gerekir.

Önce veri tabanı yaklaşımında genel olarak **Entity Data Model XML** (EDMX) dosyaları kullanılır. Bu dosyalar temel olarak mevcut bir veri tabanı üzerinden görsel olarak .NET nesneleri arasındaki ilişkiyi tanımlayan bir XML belgesidir. Visual Studio içinde **.edmx** dosyası açıldığında genellikle bir grafik **tasarım** (designer) ekranı ile gösterilir. Burada tablolar, ilişkiler ve **varlıklar** (entity) sürükle-bırak ile düzenlenebilir. Yeni projelerde genellikle Entity Framework Core tercih edilir ve EF Core, **.edmx** dosyasını desteklemez.

Projemizde Entity Framework'ü projemize dahil etmek için *Tools → NuGet Package Manager → Package Manager Console* üzerinden komut vererek bileşenleri projemize dahil ederiz

SQLite veritabanı için;

```
Install-Package Microsoft.Entityframeworkcore
Install-Package Microsoft.Entityframeworkcore.Sqlite
```

DbContext, kullanmayı öğrenmemiz gereken ilk ve en önemli EF veri tipidir. Veri tabanı sorguları için bir ara yüz görevi görür ve nesnelerde yaptığınız değişiklikleri takip eder, böylece veriler veri tabanında kalıcı hale getirilebilir.

Nesnelerini kullanarak verileri sorgulamak, eklemek, güncellemek ve silmek için EF kullanmak amacıyla öncelikle modelinizde tanımlanan varlıkları (BEC) ve ilişkileri bir veri tabanındaki tablolara eşleyen bir model (PDC ve BEC) oluşturmanız gerekir. Aşağıdaki **Öğrenci** varlığını ele alalım;

```
public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public int Yaş { get; set; }
}
```

Bir modeliniz olduğunda, iş varlığınızın uygulamanızın etkileşime girdiği ilk sınıf **System.Data.Entity.DbContext** sınıfıdır. Bu sınıf genellikle bağlam sınıfı olarak adlandırılır. Bağlam sınıfını şu amaçlarla kullanabilirsiniz:

- Sorguları yazıp çalıştırabilirsiniz.
- Sorgu sonuçlarını varlık nesneleri olarak somutlaştırabilirsiniz
- Varlık nesnelerinde yapılan değişiklikleri izleyebilirsiniz.
- Nesne değişikliklerini veri tabanında kalıcı hale getirebilirsiniz.
- Nesneleri bellekte UI denetimlerine bağlayabilirsiniz.

Bağlamla çalışmanın önerilen yolu, **DbContext** sınıfından türetilen bağlamda belirtilen varlıkların koleksiyonlarını temsil eden **DbSet** özelliklerini açığa çıkaran bir sınıf tanımlamaktır. EF Designer (EDMX) ile çalışıyorsanız, bağlam sizin için üretilecektir. Önce kod yöntemine bağlamı programcı yazar;

```
public class OkulContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }
    public DbSet<Bölüm> Bölümler { get; set; }
}
```

Bir bağlamanız olduğunda, bu özellikler (**DbSet**) aracılığıyla bağlamdaki varlıkları sorgulayabilir, ekleyebilir veya silebilirsiniz. Bu bağlam nesnesindeki **DbSet** özelliğine eriştiğinizde belirtilen tipteki tüm varlıkları döndüren bir başlangıç sorgusunu çalışır. Bu sorgu sonucu;

- Numaralandırılan nesne listesi **foreach** ile gezilebilir.
- Numaralandırılan nesne listesi **ToArray()**, **ToDictionary()** ve **ToList()** gibi yöntemlerle koleksiyonlara çevrilebilir.
- **First** veya **Any** gibi LINQ genişleme yöntemleri ile sorgulanabilir.
- Belirtilen anahtara sahip bir varlık bağlamda yüklü değilse **DbEntityEntry.Reload**, **Database.ExecuteSqlCommand** ve **DbSet<T>.Find()** ile yeniden yüklenebilir.

Bağlam nesnesinin ömrü, örneğin oluşturulmasıyla başlar ve örneğin atılması veya çöp toplanmasıyla sona erer. Bunun için **using** anahtar kelimesi kullanılır;

```
using (var context = new AppDbContext())
{
    var öğrenciler = context.Öğrenciler;
    foreach (var o in öğrenciler)
    {
        Console.WriteLine($"Ad: {o.Ad}, Yaş: {o.Yaş}");
    }
}
```

Önce Kod

Önce kod (code first) yaklaşımında ise geliştirici, modeli (PDC ve BEC) belirtmek için kod yazar. EF, geliştirici tarafından kodlanan varlık sınıflarına ve ek model yapılandırmasına bağlı olarak ver modellerini ve veri tabanı eşlemelerini **çalışma zamanında** (run time) üretir. Kısaca kodlanan varlık sınıflarından EF, veri tabanını otomatik olarak oluşturulur. Yani **eşleme** (mapping) otomatik olarak yapılır.

Bu yaklaşımda veri tabanının tasarlandığı **.edmx** dosyası kullanmadan eşleme yapmamıza olanak tanır. Ayrıca mevcut veri tabanı tablolarını da kullanabilirsiniz, buna **mevcut veri tabanı ile önce kod** (**code first with existing database**) denir.

Aşağıdaki gibi bir BEC bileşenimiz olsun;

```
public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int Yaş { get; set; }
}
```

Şimdi varlık sınıfımız ve veri tabanımız arasındaki köprü olan bağlamınız olan **DbContext** sınıfı tasarlayacağız. Bu sınıfa da varlık sınıfına ait bir **DbSet<>** özelliği eklenir;

```
public class AppDbContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }
}
```

Artık veri tabanına ilk bağlandığımız anda **Öğrenci** tablosu veri tabanında oluşacaktır; Bunu yapmak için aşağıdaki program yazılabilir,

```
using (var context = new AppDbContext())
{
    var yeniÖğrenciler = new[]
    {
        new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
        new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
        new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
    };
    context.Öğrenciler.Add(yeniÖğrenciler[0]);
    context.Öğrenciler.Add(yeniÖğrenciler[1]);
    context.Öğrenciler.Add(yeniÖğrenciler[2]);
    // ya da context.Öğrenciler.AddRange(yeniÖğrenciler);
    context.SaveChanges();
}
```

Bu program bu haliyle çalışmaz. Çünkü veri tabanı bağlantı kısmı eksiktir. Ama genel eşleşme iskeleti bu şekildedir. SQLite veri tabanı kullanarak buna bir örnek verelim. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

EFSQLiteUygulama	→ Çözüm (Solution)
└─ CodeFirst1	→ Proje (Console Application)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **EFSQLiteUygulama** adlı çözüm içinde **CodeFirst1** adlı ilk projemizi *Console Application* olarak oluşturulur.
2. Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır **Microsoft.EntityFrameworkCore** ve **Microsoft.EntityFrameworkCore.Sqlite** paketleri yüklenir.
3. Proje ile oluşan **Program.cs** dosyasını aşağıdaki şekilde değiştirilir;

```
// See https://aka.ms/new-console-template for more information
using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.Sqlite; // Install Nuget package

using (var context = new AppDbContext())
{
    context.Database.EnsureCreated();
    Console.WriteLine("SQLite Veri Tabanı Üzerinde Tablolar Oluşturuldu!");
    var yeniÖğrenciler = new[]
```

```

{
    new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
    new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
    new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
};
context.Öğrenciler.AddRange(yeniÖğrenciler);
context.SaveChanges();

var öğrenciler = context.Öğrenciler;
Console.WriteLine("Veritabanındaki Öğrenciler:");
foreach (var o in öğrenciler)
{
    Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
}

var öğrenci = context.Öğrenciler.FirstOrDefault(u => u.Ad == "İlhan");
if (öğrenci != null)
{
    context.Öğrenciler.Remove(öğrenci);
    context.SaveChanges();
    Console.WriteLine($"'{öğrenci.Ad}' adlı öğrenci silindi.");
}
context.Database.CloseConnection();
//context.Database.EnsureDeleted();
}

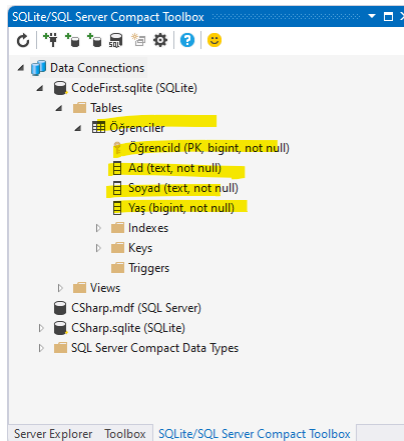
public class AppDbContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlite(@"Data Source=..\..\..\CodeFirst.sqlite");
    }
}

public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int Yaş { get; set; }
}

```

4. Program çalışıp `context.Database.EnsureCreated();` icra edildiği anda SQLite veri tabanında **Öğrenciler** tablosu oluşur;



Şekil 67. SQLite Veri Tabanı Önce Kod Tabloları

5. Aynı programa bir başka sınıf olan **User** sınıfı eklenebilir. Yeni programda **AppDbContext** nesnesi **DbContextOptionsBuilder** parametresi ile başka şekilde başlatılmıştır.

```
// See https://aka.ms/new-console-template for more information
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Sqlite;

const string databaseFile = @"..\..\..\CSharp.sqlite";
var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlite($"Data Source={databaseFile}")
    .Options;
using (var context = new AppDbContext(options))
{
    context.Database.EnsureCreated();
    Console.WriteLine("SQLite Veri Tabanı Üzerinde Tablolar Oluşturuldu!");

    var newUser = new User { Ad = "İlhan", Yaş = 30 };
    context.Users.AddRange(newUser);
    context.SaveChanges();

    var users = context.Users;
    Console.WriteLine("Veritabanındaki Kullanıcılar:");
    foreach (var u in users)
    {
        Console.WriteLine($"Id: {u.Id}, Adı: {u.Ad}, Yaşı: {u.Yaş}");
    }
    var user = context.Users.SingleOrDefault(u => u.Ad == "İlhan");
    if (user != null)
    {
        context.Users.Remove(user);
        context.SaveChanges();
        Console.WriteLine($"'{user.Ad}' adlı kullanıcı silindi.");
    }

    var yeniÖğrenciler = new[]
    {
        new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
        new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
        new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
    };
    context.Öğrenciler.AddRange(yeniÖğrenciler);
    context.SaveChanges();
    var öğrenciler = context.Öğrenciler;
    Console.WriteLine("Veritabanındaki Öğrenciler:");
    foreach (var o in öğrenciler)
    {
        Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
    }
    var öğrenci = context.Öğrenciler.FirstOrDefault(u => u.Ad == "İlhan");
    if (öğrenci != null)
    {
        context.Öğrenciler.Remove(öğrenci);
        context.SaveChanges();
        Console.WriteLine($"'{öğrenci.Ad}' adlı öğrenci silindi.");
    }
    Console.WriteLine("Sildikten Sonra Veritabanındaki Öğrenciler:");
    foreach (var o in öğrenciler)
    {
        Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
    }
    context.Database.CloseConnection();
}
```

```

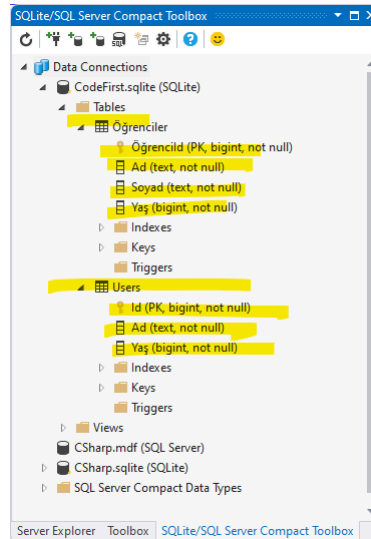
}
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<User> Users { get; set; }

    public DbSet<Öğrenci> Öğrenciler { get; set; }
}
public class User
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public int Yaş { get; set; }
}
public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int Yaş { get; set; }
}

```

6. Program çalışıp `context.Database.EnsureCreated();` icra edildiği anda SQLite veri tabanında **Öğrenciler** ve **Users** tablosu oluşur;



Şekil 68. SQLite Veri Tabanında Entity Framework Tabloları

Burada oluşan tabloların isimleri `AppDbContext` sınıfında tanımlanan `DbSet<>` özelliklerinden gelmiştir. Ayrıca veri tabanı incelendiğinde `ÖğrenciId` ve `Id` kolonlarının **birincil anahtar** (**primary key**) olduğu da görülmektedir. Bu alanlar `User` sınıfındaki `Id` ve `Öğrenci` sınıfındaki `ÖğrenciId` özelliklerinin “Id” kelimesi içeriyor olmasından kaynaklanmaktadır. Sınıfları tasarlarlarken göz önüne alınan bunun gibi birçok **kural** (**convention**) vardır.

Önce Kod Kuralları

Var Olan Bir Kuralı Geçersiz Kılma

`System.Data.Entity.ModelConfiguration.Conventions` ile tanımlanmış herhangi bir kuralı, `OnModelCreating` yöntemini geçersiz kılarak kaldırabilirsiniz.

```

public class DersContext : DbContext
{
    public DbSet<Saat> Saatler { set; get; }
    protected override void OnModelCreating(DbModelBuilder modelBuilder)
    {

```

```

modelBuilder.Conventions.Remove<PluralizingTableNameConvention>();
// PluralizingTableNameConvention adlı kural kaldırılmıştır.
}
}

```

Birincil Anahtar Kuralı

Yukarıdaki örnekte olduğu gibi varsayılan olarak, bir sınıftaki bir özellik "ID" (büyük/küçük harfe duyarlı değil) olarak adlandırılmışsa veya sınıf adı "ID" ile takip edilmişse, bu özellik birincil anahtardır. Birincil anahtar özelliğinin türü sayısal veya GUID olabilir;

```

public class Okul
{
    public int okulId { get; set; } // Primary key
    //...
}

```

Veri Tipi Keşfi

Bağlam (context) sınıfında DbSet özelliği olarak tanımlanan tipler, varlık tiplerinin (BEC) içerdiği referans tipler ve yalnızca temel sınıf DbSet özelliği olarak tanımlanmış olsa bile türetilmiş sınıflar için veri tipini otomatik olarak tanır. Buna ilişkin bir örnek SQL Server üzerinden verilecektir. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

EFSQLUygulama	→ Çözüm (Solution)
└─ CodeFirst2	→ Proje (Console Application)
├─ DataBase1.mdf	→ MSSQL Veritabanı
├─ DataBase1.ldf	→ MSSQL Veritabanı (Log)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek EFSQLUygulama adlı çözüm içinde CodeFirst2 adlı ilk projemizi *Console Application* olarak oluşturulur.
2. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New Item...* seçilir ve **Database1.mdf** adlı *Service-based Database* adlı boş veri tabanını projemize dahil edilir.
3. Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır ve **Microsoft.EntityFrameworkCore** ile **Microsoft.EntityFrameworkCore.SqlServer** paketleri yüklenir.
4. Proje ile oluşan **Program.cs** dosyasını aşağıdaki şekilde değiştirilir;

```

// See https://aka.ms/new-console-template for more information
using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.SqlServer; // Install Nuget package

string connectionString = @"Server=(LocalDB)\MSSQLLocalDB;";
connectionString += @"AttachDbFilename=D:\CSharp\EntityFramework\CodeFirst2\Database1.mdf;";
connectionString += "Integrated Security=True;Connect Timeout=30;";

var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlServer(connectionString)
    .Options;

using (var context = new AppDbContext(options))
{
    context.Database.EnsureCreated();
    Console.WriteLine("SQL Veri Tabanı Üzerinde Tablolar Oluşturuldu!");

    var yerleşkeler = context.Yerleşkeler;
    Console.WriteLine("Yerleşkeler:");
    foreach (var y in yerleşkeler)
    {
        Console.WriteLine($"Id: {y.Id}, Yerleşke Adı: {y.YerleşkeAdı}:");
        foreach (var b in y.Birimler)

```

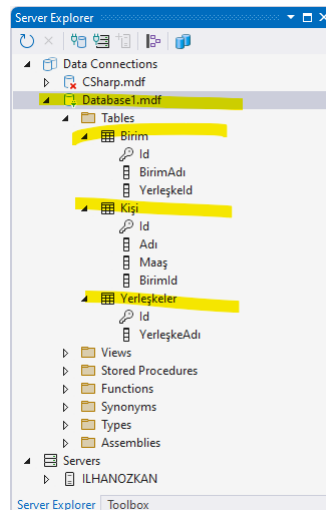


```

        {
            Console.WriteLine($"Id: {b.Id}, Birim Adı: {b.BirimAdı}:");
        }
    }
    context.Database.CloseConnection();
}
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
    public DbSet<Yerleşke> Yerleşkeler { set; get; }
}
public class Yerleşke
{
    public int Id { set; get; }
    public string YerleşkeAdı { set; get; }
    public virtual ICollection<Birim> Birimler { set; get; }
}
public class Birim
{
    public int Id { set; get; }
    public string BirimAdı { set; get; }
    public virtual ICollection<Kişi> Personel { set; get; }
}
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    public decimal Maaş { set; get; }
}
public class Müdür : Kişi
{
    public string MüdürÖzelliği { set; get; }
}
public class Şef : Kişi
{
    public string ŞefÖzelliği { set; get; }
}
public class Muhasebeci : Kişi
{
    public string MuhasebeciÖzelliği { set; get; }
}

```

5. Program çalıştığında oluşacak veri tabanı içeriği aşağıda verilmiştir;



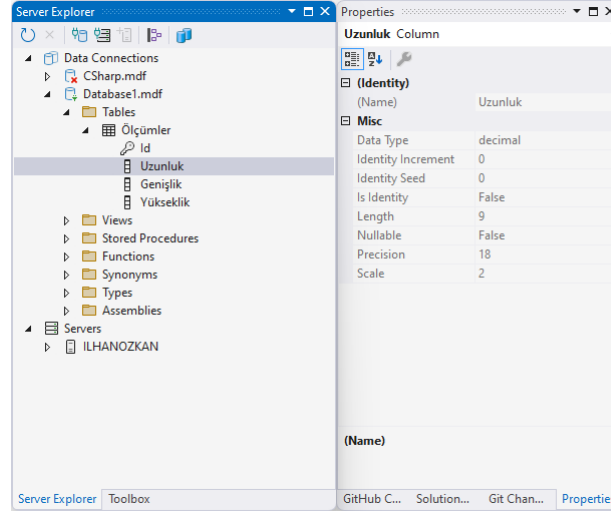
Şekil 69. SQL Veri Tabanında Entity Framework Tabloları

Ondalık Kuralı

Varsayılan olarak Entity Framework, veri tabanı tablolarındaki ondalık özellikleri tam kısmını 18 basamak, ondalık kısmı 2 basamak olacak şekilde sütunlarına eşler.

```
public class Ölçüm
{
    public int Id { set; get; }
    public decimal Uzunluk { set; get; }
    public decimal Genişlik { set; get; }
    public decimal Yükseklik { set; get; }
}
```

Yukarıdaki sınıfın veri tabanındaki karşılığı aşağıda verilmiştir;



Şekil 70. Entity Framework Ondalık Kuralı

Ondalık özelliklerin hassasiyetini aşağıdaki şekilde değiştirebiliriz:

```
protected override void OnModelCreating(DbModelBuilder modelBuilder)
{
    modelBuilder.Entity<Ölçüm>().Property(b => b.Uzunluk).HasPrecision(5, 3);
}
//Ya da
protected override void OnModelCreating(DbModelBuilder modelBuilder)
{
    modelBuilder.Conventions.Remove<DecimalPropertyConvention>();
    modelBuilder.Conventions.Add(new DecimalPropertyConvention(5, 3));
}
```

İlişki Kuralı

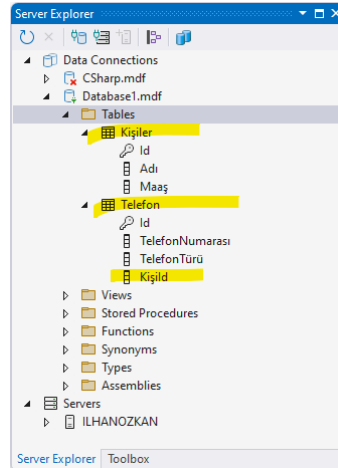
İki varlık arasındaki ilişki, **gezinme** (navigation) kuralını kullanarak otomatik olarak yapılır. Bu gezinme özelliği basit bir referans tipi veya koleksiyon tipi olabilir.

```
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    public decimal Maaş { set; get; }

    //Telefon için gezinme(navigability) özelliği
    public ICollection<Telefon> Telefonlar { set; get; }
}
public class Telefon
{
    public int Id { set; get; }
    public string TelefonNumarası { set; get; }
```

```
public string TelefonTürü { set; get; }
}
```

Yukarıdaki sınıfların veri tabanındaki karşılığı aşağıda verilmiştir;



Şekil 71. Entity Framework Gezinme Kuralı

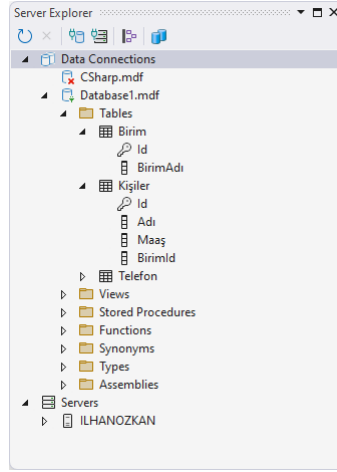
Yabancı Anahtar Kuralı

Eğer A sınıfı B sınıfıyla ilişkiliyse ve B sınıfı, A'nın birincil anahtarıyla aynı ada ve tipe sahip bir özelliğe sahipse, Entity Framework otomatik olarak bu özelliğin **yabancı anahtar** (**foreign key**) olduğunu varsayar.

```
public class Birim
{
    public int Id { set; get; }
    public string BirimAdı { set; get; }
    public virtual ICollection<Kişi> Personel { set; get; }
}
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    public decimal Maaş { set; get; }
    public ICollection<Telefon> Telefonlar { set; get; }
    public int BirimId { set; get; }
    public virtual Birim Birim { set; get; }
}
public class Telefon
{
    public int Id { set; get; }
    public string TelefonNumarası { set; get; }
    public string TelefonTürü { set; get; }
}
```

Yukarıdaki sınıfların veri tabanındaki karşılığı aşağıdaki şekilde verilmiştir. Server Explorer üzerinde Kişiler tablosu sağ tıklanıp *Open Table Definition* seçildiğinde tablonun SQL kodu aşağıdaki gibi olduğu görülecektir;

```
CREATE TABLE [dbo].[Kişiler] (
    [Id] INT IDENTITY (1, 1) NOT NULL,
    [Adı] NVARCHAR (MAX) NOT NULL,
    [Maaş] DECIMAL (18, 2) NOT NULL,
    [BirimId] INT NOT NULL,
    CONSTRAINT [PK_Kişiler] PRIMARY KEY CLUSTERED ([Id] ASC),
    CONSTRAINT [FK_Kişiler_Birim_BirimId] FOREIGN KEY ([BirimId]) REFERENCES [dbo].[Birim]
    ([Id]) ON DELETE CASCADE
);
```



Şekil 72. Entity Framework Yabancı Anahtar Kuralı

Önce Kod Veri Açıklamaları

Veri açıklamaları (**data annotations**), Entity Framework'ü veri tabanında tabloları ve kolonları oluştururken ve işlerken yönlendirir.

[Column] Niteliği

Entity Framework'e, özelliğin adını kullanmak yerine belirli bir sütun adını kullanmasını söyler.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
    public int Id { set; get; }
    [Column("Adı ve Soyadı")] //Kolon adı Adı veyine "Adı ve Soyadı" olsun
    public string Adı { set; get; }
}
```

[Required] Niteliği

Bir etki alanı sınıfının (PDC) bir özelliğine uygulandığında, veri tabanı sütunu **NOT NULL** olarak oluşturacaktır.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
    public int Id { set; get; }
    [Required] //Adı Girilmeden Kişiye kayıt atılmaz!.
    public string Adı { set; get; }
}
```

[MaxLength] ve [MinLength] Nitelikleri

[MaxLength(int)] veya [MinLength(int)] niteliği bir etki alanı sınıfının (PDC) **dizgi** (**string**) veya dizi türü özelliğine uygulanabilir. Entity Framework bir sütunun boyutunu belirtilen değere ayarlayacaktır.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
    public int Id { set; get; }
    [MinLength(3), MaxLength(100)] //Adı enz 3 krakter en fazla 100 karakter olabilir!.
    public string Adı { set; get; }
}
```

[ForeignKey(string)] Niteliği

Aşağıdaki Örneği ele alalım;

```
using System.ComponentModel.DataAnnotations.Schema;
```

```

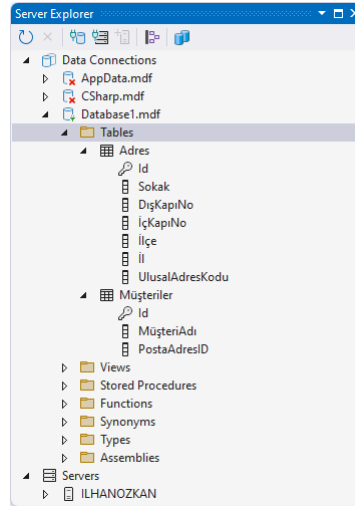
public class Müşteri
{
    public int Id { get; set; }
    [Required, MaxLength(100)]
    public string MüşteriAdı { get; set; }
    public int PostaAdresID { get; set; }
    [ForeignKey("PostaAdresID")]
    public virtual Adres PostaAdresi { get; set; }
}

public class Adres
{
    public int Id { get; set; }
    public string Sokak { get; set; }
    public string DışKapıNo { get; set; }
    public string İçKapıNo { get; set; }
    public string İlçe { get; set; }
    public string İl { get; set; }
    public long UlusalAdresKodu { get; set; }
}

```

ForeignKey nitelikleri olmadan, EF bunları karıştırabilir ve **PostaAdresi** alırken başka ID içeren değerini kullanabilir. Bunu önlemek için bu nitelik kullanılır.

Yukarıdaki örneğe ilişkin veri tabanı aşağıda verilmiştir;



Şekil 73. Entity Framework ForeignKey Niteliği

[InverseProperty(string)] Niteliği

InverseProperty, iki varlık arasında birden fazla iki yönlü ilişki olduğunda iki yönlü ilişkileri tanımlamak için kullanılabilir. Entity Framework'e diğer taraftaki özelliklerle hangi **gezinme** (**navigation**) özelliklerini eşleştirmesi gerektiğini söyler. EF, iki varlık arasında birden fazla çift yönlü ilişki olduğunda diğer taraftaki hangi gezinme özelliğinin hangi özelliklerle eşleştiğini bilmez. İlgili sınıftaki karşılık gelen gezinme özelliğinin adını parametresi olarak alması gerekir.

```

using System.ComponentModel.DataAnnotations; //gerekli
public class Birim
{
    public int Id { set; get; }
    public string BirimAdı { set; get; }
    public virtual ICollection<Kişi> AsılPersonel { set; get; }
    public virtual ICollection<Kişi> GörevliPersonel { set; get; }
}

public class Kişi
{

```

```

public int Id { set; get; }
public string Adı { set; get; }
[InverseProperty("AsılPersonel")]
public virtual Birim AnaBirim { get; set; }
[InverseProperty("GörevliPersonel")]
public virtual Birim GörevliOlduğuBirim { get; set; }
}

```

Ağaç yapısında nesneler için bu nitelik kullanılabilir;

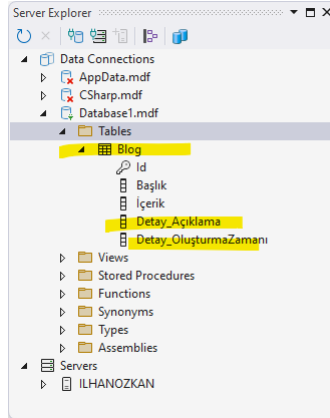
```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
public class AğaçDüğüümü
{
    public int ID { get; set; }
    public int ParentID { get; set; }
    [ForeignKey("ParentID")]
    public AğaçDüğüümü EbeveynDüğüüm { get; set; }
    [InverseProperty("ParentNode")]
    public virtual ICollection<AğaçDüğüümü> ÇocukDüğüümler { get; set; }
}

```

[ComplexType] Niteliği

Alan odaklı tasarımda (Domain Driven Design-DDD) değer nesneleri (value object) gibi karmaşık tipler kendi başlarına izlenemezler ancak bir varlığın parçası olarak izlenirler. Böyle durumda bu nitelik kullanılır.



Şekil 74. Entity Framework ComplexType Niteliği

Veri tabanını oluşturan kod içeriği aşağıda verilmiştir;

```

[ComplexType]
public class BlogDetay
{
    public DateTime? OluşturmaZamanı { get; set; }
    [MaxLength(250)]
    public string Açıklama { get; set; }
}
public class Blog
{
    public int Id { get; set; }
    public string Başlık { get; set; }
    public string İçerik { get; set; }
    public BlogDetay Detay { get; set; }
}

```

[Range(min,max)] Niteliği

İş varlığına ilişkin sayısal alanlarda sınır belirlemek için kullanılır;

```

public class MezuniyetDerecesi
{
    public int DereceID { get; set; }
    [Range(0, 4)]
    public Nullable<decimal> Derece { get; set; }
}

public class Ogrenci
{
    public int Id { get; set; }
    [Required, MaxLength(50)]
    public string Adı { get; set; }
    [Required, MaxLength(50)]
    public string Soyadı { get; set; }
    [EmailAddress, MaxLength(150)]
    public string EPosta { get; set; }
    public MezuniyetDerecesi MezuniyetDerecesi { get; set; }
}

```

[NotMapped] Niteliği

Önce kod yaklaşımında, Entity Framework, desteklenen bir veri türünde olan her genel özellik için bir sütun oluşturur. **[NotMapped]** niteliği, bir veri tabanı tablosunda sütun istemediğimiz tüm özelliklere uygulanmalıdır.

```

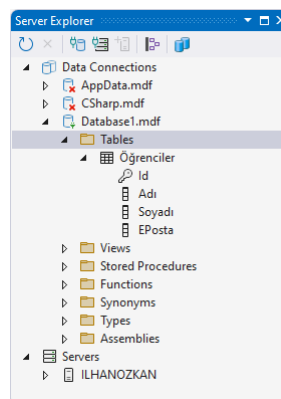
public class Öğrenci
{
    public int Id { get; set; }
    [Required, MaxLength(50)]
    public string Adı { get; set; }
    [Required, MaxLength(50)]
    public string Soyadı { get; set; }

    public string TamAdı => string.Format("{0} {1}", Adı, Soyadı);

    [Required, MaxLength(150)]
    public string EPosta { get; set; }
    [NotMapped]
    public float Ortalama { get; set; }
}

```

Oluşan veri tabanı aşağıda verilmiştir;



Şekil 75. Entity Framework NotMapped Niteliği

[Table] Niteliği

İş varlığına ilişkin veri tabanında tablo adı belirlemek için kullanılır.

```

[Table("Personel")]
public class Kişi

```

```
{
    public int Id { set; get; }
    public string Adı { set; get; }
}
```

[Index] Niteliği

Veri tabanında oluşan kolonlar için **dizin** (index) oluşturmak için kullanılır;

```
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }

    [Index]
    public int Yaş { set; get; }
}
```

İstenirse dizine ad da verilebilir;

```
[Index("IX_Kişi_Yaş")]
public int Yaş { set; get; }
```

İstenirse **tekil** (unique) dizin de oluşturulabilir;

```
[Index(IsUnique = true)]
public int Yaş { set; get; }
```

Ayrıca birden fazla kolon kullanılarak da dizin oluşturulabilir;

```
[Index("IX_Kişi_AdveYaş",1)]
public string Adı { set; get; }
[Index("IX_Kişi_AdveYaş ",2)]
public int Yaş { set; get; }
```

[Key] Niteliği

Anahtar (key), bir veri tabanı tablosundaki her satırı/kaydı benzersiz bir şekilde tanımlayan bir alandır. Varlık sınıfında "ID" içeren bir özellik yoksa bu nitelik kullanılabilir;

```
public class Personel
{
    [Key]
    public int SicilNo { set; get; }
    public string Adı { set; get; }
}
```

Aslında bu niteliği kullanmadan aşağıdaki gibi yapılabilirdi;

```
public class Personel
{
    public int PersonelId { set; get; } //Personel ID anahtar olarak görülür
    public string Adı { set; get; }
}
```

[StringLength(int)] Niteliği

Varlık nesnesinde metinlere karakter sınırlaması getirmek için kullanılır;

```
public class Personel
{
    public int PersonelId { set; get; }

    [StringLength(100)]
    public string Adı { set; get; }
}
```


[DatabaseGenerated] Niteliği

Bazı durumlarda anahtar alanın otomatik artan bir kolon olması istenmez. Bunun gibi durumlarda bu nitelik kullanılabilir;

```
public class İl
{
    public int İlId { get; set; } // İlId veritabanında otomatik artıtılan (IDENTITY)
    public string İlAdı { get; set; }
}
```

Eğer il trafik kodlarını **anahtar** (key) olarak otomatik artmayan bir sütun istiyorsak varlığı aşağıdaki şekilde tanımlamalıyız;

```
public class İl
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.None)]
    public int TrafikKodu { get; set; }
    public string İlAdı { get; set; }
}
```

Veri tabanında hesaplanan bir kolon varsa da bu nitelik kullanılabilir. Aşağıdaki gibi bir tablo oluşturmak istiyorsak;

```
CREATE TABLE [Personel] (
    [Adı] varchar(100) PRIMARY KEY,
    [DoğumTarihi] Date NOT NULL,
    [Yaş] AS DATEDIFF(year, [DoğumTarihi], GETDATE())
)
```

Bu varlığı aşağıdaki gibi kodlamalıyız;

```
[Table("Personel")]
public class Kişi
{
    [Key, StringLength(100)]
    public string Adı { get; set; }
    public DateTime DoğumTarihi { get; set; }

    [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
    public int Yaş { get; set; }
}
```

Burada çok kullanılan niteliklere yer verilmiştir³⁹.

Mevcut Veri Tabanı Tablolarıyla Çalışma

Önce kod (code first) yaklaşımıyla hali hazırda var olan tablolar içeren veri tabanı ile çalışılabilir. Bunun için içerisinde il listesi olan bir veri tabanını (Database2.mdf) örnek alalım. Veri tabanında aşağıdaki tablo var olsun;

```
CREATE TABLE [dbo].[İller] (
    [İlTrafikKodu] INT NOT NULL,
    [İlAdı] NVARCHAR (MAX) NOT NULL,
    CONSTRAINT [PK_İller] PRIMARY KEY CLUSTERED ([İlTrafikKodu] ASC)
);
```

Bu tablonun içinde de il listemiz işlemiş olsun. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

EFSQLUygulama	→ Çözüm (Solution)
└ CodeFirst3	→ Proje (Console Application)
└─ DataBase2.mdf	→ MSSQL Veritabanı

³⁹ <https://learn.microsoft.com/en-us/ef/ef6/modeling/code-first/data-annotations>



DataBase2.ldf
Program.cs

→ MSSQL Veritabanı (Log)
→ Ana (Main) Program

Bu durumda aşağıdaki kod ile veri tabanını sorgulayabiliriz;

```
// See https://aka.ms/new-console-template for more information using
Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

string connectionString = @"Server=(LocalDB)\MSSQLLocalDB;";
connectionString += @"AttachDbFilename=D:\CSharp\EntityFramework\CodeFirst3\Database2.mdf;";
connectionString += "Integrated Security=True;Connect Timeout=30;";

var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlServer(connectionString)
    .Options;

using (var context = new AppDbContext(options))
{
    var il=context.İller.FirstOrDefault();
    if (il != null)
    {
        Console.WriteLine($"İl: {il.İlAdı}, Trafik Kodu: {il.İlTrafikKodu}");
        //İl: Adana, Trafik Kodu: 1
    }
    else
    {
        Console.WriteLine("İl bulunamadı.");
    }
    il.İlAdı="Adana 1 Numara";
    context.SaveChanges();
    context.Database.CloseConnection();
}

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
    public DbSet<İl> İller { set; get; }
}

public class İl
{
    [Key]
    public int İlTrafikKodu { get; set; }
    public string İlAdı { get; set; }
}
```

Veri Göçü

Gerçek olarak uygulanan projelerde, özellikler uygulandıkça veri modelleri değişir: yeni varlıklar veya özellikler eklenir ve kaldırılır ve veri tabanı şemalarının uygulama ile senkronize tutulabilmesi için buna göre değiştirilmesi gerekir.

EF Core'daki **göç** (migration) özelliği, veri tabanındaki verileri korurken veri tabanı şemasını uygulamanın veri modeliyle senkronize tutmak için artımlı olarak güncelleme bir yolunu sağlar.

EF Core'daki **göç** özelliğini kullanabilmek için; EF tarafından yönetilmiş veri tabanı nesnelerinizi içeren geçerli bir **DbContext** içeren bir uygulamanız olması gerekir. Bunun için bir proje oluşturalım. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

EFSQLUygulama

→ Çözüm (Solution)

└─ CodeFirst4	→ Proje (Console Application)
└─ DataBaseEF.mdf	→ MSSQL Veritabanı
└─ DataBaseEF.ldf	→ MSSQL Veritabanı (Log)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **EFSQLUygulama** adlı çözüm içinde **CodeFirst4** adlı ilk projemizi *Console Application* olarak oluşturulur.
2. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New Item...* seçilir ve **DatabaseEF.mdf** adlı *Service-based Database* adlı boş veri tabanını projemize dahil edilir.
3. Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır **Microsoft.EntityFrameworkCore** ve **Microsoft.EntityFrameworkCore.SqlServer** paketleri yüklenir.
4. Oluşan projede **Program.cs** aşağıdaki gibi değiştirilir ve bir kez çalıştırılarak veri tabanında ilgili tablolar ve verilerin oluşması sağlanır.
5. Burada dikkat edilmesi gereken **AppDbContext** sınıfının yapıcısının parametre almamasını sağlamaktır. Bunu yapmak için **onConfiguring()** yöntemi bu sınıfa eklenir. Ayrıca il listesini de göç sırasında oluşturmak istiyorsak **OnModelCreating()** yöntemini de ekleyip yeni verileri göç edilen veri tabanına kaydedebiliriz.

```
// See https://aka.ms/new-console-template for more information
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

using (var context = new AppDbContext())
{
    context.Database.EnsureCreated();
    var iller = context.İller.ToList();
    Console.WriteLine("İller:");
    foreach (var il in iller)
    {
        Console.WriteLine($"Trafik Kodu: {il.İlTrafikKodu}, İl Adı: {il.İlAdı}");
    }
    context.Database.CloseConnection();
}

public class AppDbContext : DbContext
{
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        string connectionString = @"Server=(LocalDB)\MSSQLLocalDB;";
        connectionString += @"AttachDbFilename=D:\CSharp\EntityFramework\CodeFirst4\";
        connectionString += "DatabaseModel.mdf;";
        connectionString += "Integrated Security=True;";
        connectionString += "Connect Timeout=30;";
        optionsBuilder.UseSqlServer(connectionString);
    }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Personel>()
            .HasOne(p => p.Adres)
            .WithMany()
            .HasForeignKey(p => p.PostaAdresID);
        modelBuilder.Entity<Adres>()
            .HasOne(a => a.İl)
            .WithMany()
            .HasForeignKey(a => a.İlTrafikKodu);
        modelBuilder.Entity<İl>().HasData(
            new İl { İlTrafikKodu = 1, İlAdı = "Adana" },
            new İl { İlTrafikKodu = 2, İlAdı = "Adıyaman" },
            new İl { İlTrafikKodu = 3, İlAdı = "Afyonkarahisar" },

```

```

new İl { İlTrafikKodu = 4, İlAdı = "Ağrı" },
new İl { İlTrafikKodu = 5, İlAdı = "Amasya" },
new İl { İlTrafikKodu = 6, İlAdı = "Ankara" },
new İl { İlTrafikKodu = 7, İlAdı = "Antalya" },
new İl { İlTrafikKodu = 8, İlAdı = "Artvin" },
new İl { İlTrafikKodu = 9, İlAdı = "Aydın" },
new İl { İlTrafikKodu = 10, İlAdı = "Balıkesir" },
new İl { İlTrafikKodu = 11, İlAdı = "Bilecik" },
new İl { İlTrafikKodu = 12, İlAdı = "Bingöl" },
new İl { İlTrafikKodu = 13, İlAdı = "Bitlis" },
new İl { İlTrafikKodu = 14, İlAdı = "Bolu" },
new İl { İlTrafikKodu = 15, İlAdı = "Burdur" },
new İl { İlTrafikKodu = 16, İlAdı = "Bursa" },
new İl { İlTrafikKodu = 17, İlAdı = "Çanakkale" },
new İl { İlTrafikKodu = 18, İlAdı = "Çankırı" },
new İl { İlTrafikKodu = 19, İlAdı = "Çorum" },
new İl { İlTrafikKodu = 20, İlAdı = "Denizli" },
new İl { İlTrafikKodu = 21, İlAdı = "Diyarbakır" },
new İl { İlTrafikKodu = 22, İlAdı = "Edirne" },
new İl { İlTrafikKodu = 23, İlAdı = "Elazığ" },
new İl { İlTrafikKodu = 24, İlAdı = "Erzincan" },
new İl { İlTrafikKodu = 25, İlAdı = "Erzurum" },
new İl { İlTrafikKodu = 26, İlAdı = "Eskişehir" },
new İl { İlTrafikKodu = 27, İlAdı = "Gaziantep" },
new İl { İlTrafikKodu = 28, İlAdı = "Giresun" },
new İl { İlTrafikKodu = 29, İlAdı = "Gümüşhane" },
new İl { İlTrafikKodu = 30, İlAdı = "Hakkari" },
new İl { İlTrafikKodu = 31, İlAdı = "Hatay" },
new İl { İlTrafikKodu = 32, İlAdı = "Isparta" },
new İl { İlTrafikKodu = 33, İlAdı = "Mersin" },
new İl { İlTrafikKodu = 34, İlAdı = "İstanbul" },
new İl { İlTrafikKodu = 35, İlAdı = "İzmir" },
new İl { İlTrafikKodu = 36, İlAdı = "Kars" },
new İl { İlTrafikKodu = 37, İlAdı = "Kastamonu" },
new İl { İlTrafikKodu = 38, İlAdı = "Kayseri" },
new İl { İlTrafikKodu = 39, İlAdı = "Kırklareli" },
new İl { İlTrafikKodu = 40, İlAdı = "Kırşehir" },
new İl { İlTrafikKodu = 41, İlAdı = "Kocaeli" },
new İl { İlTrafikKodu = 42, İlAdı = "Konya" },
new İl { İlTrafikKodu = 43, İlAdı = "Kütahya" },
new İl { İlTrafikKodu = 44, İlAdı = "Malatya" },
new İl { İlTrafikKodu = 45, İlAdı = "Manisa" },
new İl { İlTrafikKodu = 46, İlAdı = "Kahramanmaraş" },
new İl { İlTrafikKodu = 47, İlAdı = "Mardin" },
new İl { İlTrafikKodu = 48, İlAdı = "Muğla" },
new İl { İlTrafikKodu = 49, İlAdı = "Muş" },
new İl { İlTrafikKodu = 50, İlAdı = "Nevşehir" },
new İl { İlTrafikKodu = 51, İlAdı = "Niğde" },
new İl { İlTrafikKodu = 52, İlAdı = "Ordu" },
new İl { İlTrafikKodu = 53, İlAdı = "Rize" },
new İl { İlTrafikKodu = 54, İlAdı = "Sakarya" },
new İl { İlTrafikKodu = 55, İlAdı = "Samsun" },
new İl { İlTrafikKodu = 56, İlAdı = "Siirt" },
new İl { İlTrafikKodu = 57, İlAdı = "Sinop" },
new İl { İlTrafikKodu = 58, İlAdı = "Sivas" },
new İl { İlTrafikKodu = 59, İlAdı = "Tekirdağ" },
new İl { İlTrafikKodu = 60, İlAdı = "Tokat" },
new İl { İlTrafikKodu = 61, İlAdı = "Trabzon" },
new İl { İlTrafikKodu = 62, İlAdı = "Tunceli" },
new İl { İlTrafikKodu = 63, İlAdı = "Şanlıurfa" },
new İl { İlTrafikKodu = 64, İlAdı = "Uşak" },
new İl { İlTrafikKodu = 65, İlAdı = "Van" },

```

```

        new İl { İlTrafikKodu = 66, İlAdı = "Yozgat" },
        new İl { İlTrafikKodu = 67, İlAdı = "Zonguldak" },
        new İl { İlTrafikKodu = 68, İlAdı = "Aksaray" },
        new İl { İlTrafikKodu = 69, İlAdı = "Bayburt" },
        new İl { İlTrafikKodu = 70, İlAdı = "Karaman" },
        new İl { İlTrafikKodu = 71, İlAdı = "Kırıkkale" },
        new İl { İlTrafikKodu = 72, İlAdı = "Batman" },
        new İl { İlTrafikKodu = 73, İlAdı = "Şırnak" },
        new İl { İlTrafikKodu = 74, İlAdı = "Bartın" },
        new İl { İlTrafikKodu = 75, İlAdı = "Ardahan" },
        new İl { İlTrafikKodu = 76, İlAdı = "Iğdır" },
        new İl { İlTrafikKodu = 77, İlAdı = "Yalova" },
        new İl { İlTrafikKodu = 78, İlAdı = "Karabük" },
        new İl { İlTrafikKodu = 79, İlAdı = "Kilis" },
        new İl { İlTrafikKodu = 80, İlAdı = "Osmaniye" },
        new İl { İlTrafikKodu = 81, İlAdı = "Düzce" }
    );
}

public DbSet<Personel> Personel { set; get; }
public DbSet<İl> İller { set; get; }
}

public class Personel
{
    public int PersonelId { get; set; }
    [StringLength(100)]
    public string Adı { get; set; }
    public string Soyadı { get; set; }
    public int PostaAdresID { get; set; }
    [ForeignKey("PostaAdresID")]
    public virtual Adres Adres { get; set; }
}

public class Adres {
    public int AdresId { get; set; }
    [StringLength(200)] //Cadde-Sokak-Bulvar-Meydan
    public string CSBM { get; set; }
    [StringLength(10)]
    public string DışKapıNo { get; set; }
    [StringLength(10)]
    public string İçKapıNo { get; set; }
    [StringLength(100)]
    public string İlçe { get; set; }
    [StringLength(50)]
    public int İlTrafikKodu { get; set; }
    [ForeignKey("İlTrafikKodu")]
    public virtual İl İl { get; set; }
    public long PostaKodu { get; set; }
}

public class İl
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.None)]
    public int İlTrafikKodu { get; set; }
    public string İlAdı { get; set; }
}

```

Bir EF Core projesinde göç (migration) işlemleri şu şekilde kullanılır;

- Bir veri modeli değişikliği yapıldığında, geliştirici veri tabanı şemasını senkronize tutmak için gerekli güncellemeleri açıklayan karşılık gelen bir göç eklemek için **EF Core Tools** kullanır. EF Core, farklılıkları belirlemek için geçerli modeli eski modelin bir anlık görüntüsüyle karşılaştırır ve göç

kaynak dosyaları oluşturur; dosyalar projenizin kaynak denetiminde diğer kaynak dosyaları gibi izlenebilir.

- Yeni bir göç oluşturulduktan sonra, çeşitli şekillerde bir veri tabanına uygulanabilir. EF Core uygulanan tüm göçleri özel bir geçmiş tablosunda kaydeder ve hangi göçlerin uygulandığını ve hangilerinin uygulanmadığını bilmesini sağlar.

Projemizde göç yapabilmek için Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır **Microsoft.EntityFrameworkCore.Tools** paketi yüklenir. Ardından rojemizde veri tabanının ve modelin ilk tapısını saklamak için; Visual Studio üzerinde *Tools→NuGet Package Manager→Package Manager Console* tıklanır ve açılan pencerede **Add-Migration Başlangıç** komutu yazılır ve çalıştırılır. Burada dikkat edilmesi gereken “Default Project” ayarının mevcut projemizi göstermesidir.

```
PM> Add-Migration Başlangıç
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

Komut sonrasında projemizde aşağıdaki klasör ve dosyalar oluşur;

EFSQLUygulama	→ Çözüm (Solution)
└─ CodeFirst4	→ Proje (Console Application)
└─ Migrations	→ Klasör
└─ 20250530122325_Başlangıç.cs	→ Klasör
└─ AppDbContextModelSnapshot.cs	→ Klasör
─ DataBaseEF.mdf	→ MSSQL Veritabanı
─ DataBaseEF.ldf	→ MSSQL Veritabanı (Log)
─ Program.cs	→ Ana (Main) Program

Oluşan dosyanın adı (**20250530083726_Başlangıç.cs**), **zaman damgası** (**time stamp**) ve göç adını içerir. Diğeri ise varlıklarımızın o anki bir fotoğrafı olan **AppDbContextModelSnapshot.cs** dosyasıdır. Şimdi göç dosyasının içeriğini inceleyelim;

```
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable
#pragma warning disable CA1814 // Prefer jagged arrays over multidimensional

namespace CodeFirst4.Migrations
{
    /// <inheritdoc />
    public partial class Başlangıç : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "İller",
                columns: table => new
                {
                    İlTrafikKodu = table.Column<int>(type: "int", nullable: false),
                    İlAdı = table.Column<string>(type: "nvarchar(max)", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_İller", x => x.İlTrafikKodu);
                });
            migrationBuilder.CreateTable(
                name: "Adres",
                columns: table => new
                {
                    AdresId = table.Column<int>(type: "int", nullable: false)
```

```

        .Annotation("SqlServer:Identity", "1, 1"),
        CSBM = table.Column<string>(type: "nvarchar(200)",
            maxLength: 200, nullable: false),
        DışKapıNo = table.Column<string>(type: "nvarchar(10)",
            maxLength: 10, nullable: false),
        İçKapıNo = table.Column<string>(type: "nvarchar(10)",
            maxLength: 10, nullable: false),
        İlçe = table.Column<string>(type: "nvarchar(100)",
            maxLength: 100, nullable: false),
        İlTrafikKodu = table.Column<int>(type: "int",
            maxLength: 50, nullable: false),
        PostaKodu = table.Column<long>(type: "bigint", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Adres", x => x.AdresId);
        table.ForeignKey(
            name: "FK_Adres_İller_İlTrafikKodu",
            column: x => x.İlTrafikKodu,
            principalTable: "İller",
            principalColumn: "İlTrafikKodu",
            onDelete: ReferentialAction.Cascade);
    });
migrationBuilder.CreateTable(
    name: "Personel",
    columns: table => new
    {
        PersonelId = table.Column<int>(type: "int", nullable: false)
            .Annotation("SqlServer:Identity", "1, 1"),
        Adı = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
            nullable: false),
        Soyadı = table.Column<string>(type: "nvarchar(max)", nullable: false),
        PostaAdresID = table.Column<int>(type: "int", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Personel", x => x.PersonelId);
        table.ForeignKey(
            name: "FK_Personel_Adres_PostaAdresID",
            column: x => x.PostaAdresID,
            principalTable: "Adres",
            principalColumn: "AdresId",
            onDelete: ReferentialAction.Cascade);
    });

migrationBuilder.InsertData(
    table: "İller",
    columns: new[] { "İlTrafikKodu", "İlAdı" },
    values: new object[,]
    {
        { 1, "Adana" },
        { 2, "Adıyaman" },
        { 3, "Afyonkarahisar" },
        { 4, "Ağrı" },
        { 5, "Amasya" },
        { 6, "Ankara" },
        { 7, "Antalya" },
        { 8, "Artvin" },
        { 9, "Aydın" },
        { 10, "Balıkesir" },
        { 11, "Bilecik" },
        { 12, "Bingöl" },
    }
);

```

```
{ 13, "Bitlis" },
{ 14, "Bolu" },
{ 15, "Burdur" },
{ 16, "Bursa" },
{ 17, "Çanakkale" },
{ 18, "Çankırı" },
{ 19, "Çorum" },
{ 20, "Denizli" },
{ 21, "Diyarbakır" },
{ 22, "Edirne" },
{ 23, "Elazığ" },
{ 24, "Erzincan" },
{ 25, "Erzurum" },
{ 26, "Eskişehir" },
{ 27, "Gaziantep" },
{ 28, "Giresun" },
{ 29, "Gümüşhane" },
{ 30, "Hakkari" },
{ 31, "Hatay" },
{ 32, "Isparta" },
{ 33, "Mersin" },
{ 34, "İstanbul" },
{ 35, "İzmir" },
{ 36, "Kars" },
{ 37, "Kastamonu" },
{ 38, "Kayseri" },
{ 39, "Kırklareli" },
{ 40, "Kırşehir" },
{ 41, "Kocaeli" },
{ 42, "Konya" },
{ 43, "Kütahya" },
{ 44, "Malatya" },
{ 45, "Manisa" },
{ 46, "Kahramanmaraş" },
{ 47, "Mardin" },
{ 48, "Muğla" },
{ 49, "Muş" },
{ 50, "Nevşehir" },
{ 51, "Niğde" },
{ 52, "Ordu" },
{ 53, "Rize" },
{ 54, "Sakarya" },
{ 55, "Samsun" },
{ 56, "Siirt" },
{ 57, "Sinop" },
{ 58, "Sivas" },
{ 59, "Tekirdağ" },
{ 60, "Tokat" },
{ 61, "Trabzon" },
{ 62, "Tunceli" },
{ 63, "Şanlıurfa" },
{ 64, "Uşak" },
{ 65, "Van" },
{ 66, "Yozgat" },
{ 67, "Zonguldak" },
{ 68, "Aksaray" },
{ 69, "Bayburt" },
{ 70, "Karaman" },
{ 71, "Kırıkkale" },
{ 72, "Batman" },
{ 73, "Şırnak" },
{ 74, "Bartın" },
```



```

        { 75, "Ardahan" },
        { 76, "İğdır" },
        { 77, "Yalova" },
        { 78, "Karabük" },
        { 79, "Kilis" },
        { 80, "Osmaniye" },
        { 81, "Düzce" }
    });
    migrationBuilder.CreateIndex(
        name: "IX_Adres_İlTrafikKodu",
        table: "Adres",
        column: "İlTrafikKodu");
    migrationBuilder.CreateIndex(
        name: "IX_Personel_PostaAdresID",
        table: "Personel",
        column: "PostaAdresID");
}
/// <inheritdoc />
protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropTable(
        name: "Personel");

    migrationBuilder.DropTable(
        name: "Adres");

    migrationBuilder.DropTable(
        name: "İller");
}
}
}

```

Gördüğünüz gibi, **Up()** yöntemi **İller**, **Adres** ve **Personel** adlı üç tablo ve alanları oluşturulur. Ek olarak, tablolar arasındaki ilişkiler ve **dizinler** (**index**) de oluşturulur. Diğer tarafta **Down()** yöntemi göç aktivitelerini (**Up** yöntemi ile yapılanları) tersine çevirmeye çalışır. Göçünüz konusunda kendinize güveniyorsanız, göçü istediğiniz veri tabanına uygulayabilirsiniz. Bunun için mevcut projemizde kullandığımız veri tabanını değiştireceğiz.

1. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New Item...* seçilir ve **DatabaseModel.mdf** adlı *Service-based Database* adlı boş veri tabanını projemize dahil edilir.
2. **Program.cs** içinde yeni veri tabanı bağlantımızı sağlayacak değişikliği **AppDbContext** sınıfının **onConfiguring()** yönteminde yapıyoruz.

```

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    string connectionString = @"Server=(LocalDB)\MSSQLLocalDB;";
    connectionString += @"AttachDbFilename=D:\CSharp\EntityFramework\CodeFirst4\";
    connectionString += "DatabaseModel.mdf;";
    connectionString += "Integrated Security=True;";
    connectionString += "Connect Timeout=30;";
    optionsBuilder.UseSqlServer(connectionString);
}

```

3. Visual Studio üzerinde *Tools→NuGet Package Manager→Package Manager Console* tıklanır ve açılan pencerede **Add-Migration Başlangıç** komutu yazılır ve çalıştırılır;

PM> **Update-Database**

Build started...

Build succeeded.

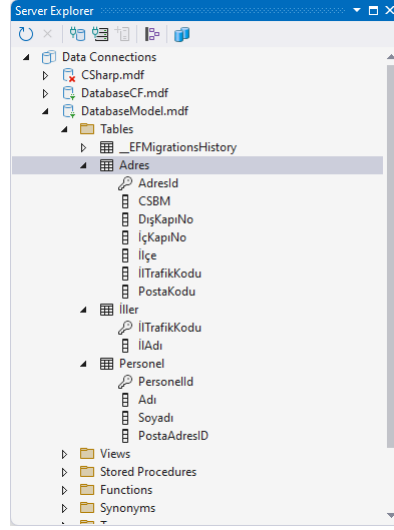
Acquiring an exclusive lock for migration application. See <https://aka.ms/efcore-docs-migrations-lock> for more information if this takes too long.

Applying migration '**20250530122325_Başlangıç**'.

Done.

PM>

Artık projemiz yeni veri tabanında tabloları oluşturmuştur;



Şekil 76. Göç Sonrası Yeni Veri Tabanı

Add-Migration ve **Update-Database** komutları için aktiviteleri ayarlamak için kullanılabilecek çeşitli seçenekler mevcuttur. Bunları görmek ve incelemek için **get-help Add-Migration** ve **get-help Update-Database** komutları kullanılabilir.

Geliştirmenin yapıldığı ortamın dışında çalışan uygulamalar genellikle veri tabanı **güncellemeleri** (**update**) gerektirir. Veri tabanı **yamalarını** (**patch**) oluşturmak için **AddMigration** komutunu kullandıktan sonra, güncellemeleri test ve diğer ortamlarda çalıştırmamız gerekir. Bunun yanında uygulamanın müşteri ortamına bağlantı izni yok ya da bu ortamda Visual Studio kurulmamış ise bu komutu çalıştıramayız.

Bunun çözümü örneğimizde olduğu gibi **DbContext** sınıfına **OnModelCreating** yöntemini eklemektir. Göç sırasında yeni veri tabanında verileri oluşturma işlemine **veri ile besleme** (**seeding data**) adı verilir.